

the hex file, click 'Address/Data' radio button (refer Fig. 4). If the file loaded is not a hex file, or the loaded hex file is erroneous, a predefined error message will be displayed in the text box.

Making standalone application

You can make a standalone application that can run on any PC without install-

ing Visual Studio 2008. In 'Project' menu, click 'Project Properties.' In 'Properties' template, click 'Publish' tab followed by 'Publish Wizard' option. Give the path for the setup program to be saved. A setup file is created, which can be used for installing the hex file parser program like any other software package. Install .NET 2.0 and run the setup program. The hex file parser ex-

ecutable file is installed and can be run in your system.

EFY note. The setup program and source code files (Hex File Parser and HexParser) of the project are included in this month's EFY-CD.

Dr H.N. Pandya is head & reader in Department of Electronics, Saurashtra University, Rajkot, and Chinutkar Harshadkumar Govindrao is a research scholar under his supervision

FILEPARSER.CS

```

/*Below is hexParser.cs file used to make
hexParser.dll*/

using System;
using System.Text;
using System.IO;
namespace hexParser
{
    public class FileParser
    {
        Int32 start, end; //used data
        range
        private UInt16 length; //record
        length, no. of bytes
        private Int32 offset; //offset
        address
        private UInt16 type; //record
        type
        private const int size_avr =
        262144; //256KBytes
        private UInt16[] data = new
        UInt16[size_avr]; //temp buffer
        public UInt16[] filedata = new
        UInt16[size_avr]; //contains array of
        instruction code byte
        public FileParser() { //fills data
        with 0xff
        for (int i = 0; i < filedata.
        Length; i++)
        {
            filedata[i] = 0xff;
        }
        public Int32 Fstart {
            get {
                return start;
            }
        }
        public Int32 Fend
        {
            get
            {
                return end;
            }
        }
        //code function below gives the
        address/data structure
        public string readFile(string
        filename)
        {
            Int32 recPoint, baseadd = 0,
            size = size_avr;
            This.start = size_avr; //flash
            size of the target AVR
            this.end = 0;
            String hexLine, dum;
            StringBuilder stb = new
            StringBuilder();
            try
            {
                using (StreamReader str
                = File.OpenText(filename))
                {
                    while ((hexLine =
                    str.ReadLine()) != null)
                    {
                        dum = this.
                        parseRecord(hexLine);
                        switch (this.
                        type)
                        {
                            case 0: //for
                            data record 00 type
                            if
                            ((baseadd + this.offset + this.length)
                            > size)
                            {
                                throw new Exception("Data Buffer limit
                                reached, May be larger than device mem-
                                ory limits" + hexLine);
                            }
                            for
                            (recPoint = 0; recPoint < this.length;
                            recPoint++)
                            {
                                /*
                                Int32 aadd = ((baseadd + this.offset) >
                                1) + recPoint;
                                */
                                //must be half of obtained address from
                                file
                                * This is for 8051 microcontrollers
                                only*/
                                Int32 aadd = (baseadd + this.offset +
                                recPoint) >> 1;
                                this.filedata[baseadd + this.offset +
                                recPoint] = this.data[recPoint];
                                // stb.AppendLine("Add - " + (aadd).
                                ToString("X2") +
                                " Data - " +
                                this.filedata[baseadd + this.offset +
                                recPoint].ToString("X2"));
                                // this.myHex.Add(aadd, this.
                                filedata[baseadd + this.offset +
                                recPoint]);
                                //update
                                start and end address
                                if ((baseadd + this.offset) < start)
                                {
                                    start = baseadd + this.offset;
                                }
                                //
                                start >= 1;
                                if ((baseadd + this.offset + this.length - 1)
                                > end)
                                {
                                    end
                                    = baseadd + this.offset + this.length
                                    - 1;
                                }
                                //
                                Extended segment address record ?
                                baseadd
                                = (this.data[0] << 8) | this.data[1];
                                baseadd
                                <<= 4;
                                break;
                                case 3: //
                                Start segment address record ?
                                break;
                                // Ignore it, since we have no influence
                                on execution start address.
                                case 4: //
                                Extended linear address record ?
                                baseadd
                                = (this.data[0] << 8) | this.data[1];
                                baseadd
                                <<= 16;
                                break;
                                case 5: //
                                Start linear address record ?
                                break;
                                // Ignore it, since we have no influence
                                on execution start address.
                                case 1: //end
                                of record ,,end of file
                                str.
                                Close();
                                return
                                "Success";
                                default:
                                //str.Close();
                                throw
                                new Exception("Wrong HEX file format " +
                                hexline);
                                }
                                str.Close();
                                }
                                catch (Exception e)
                                {
                                    return "Exception " +
                                    e.Message; // + e.StackTrace;
                                }
                                return "this must not be
                                the end";
                                //code function below parses
                                each line for hex format
                                public string parseRecord(string
                                hexLine)
                                {
                                    UInt16 checksum;
                                    int recPointer;
                                    if (hexLine.Length < 11)
                                    {
                                        throw new

```